BÀI THỰC HÀNH CHƯƠNG 5

I. Tạo một REST API đơn giản

Bài thực hành này sẽ tạo một REST API sử dụng Django REST Framework (DRF) và triển khai một điểm cuối (endpoint) API sử dụng view dạng hàm. Điểm cuối này sẽ cung cấp thông tin về tổng số sách trong CSDL.

Lưu ý: Cần thực hiện cài đặt và cấu hình REST framework như trong slide Chương 5.

1. Tạo một tập tin tên là **api_views.py** trong thư mục **bookr/reviews**.
   View REST API sẽ tương tự như các view thông thường khác của Django. Ta có thể thêm view API vào tập tin **views.py** giống như những view khác nếu muốn. Các view API trong bài này được đưa vào một tập tin riêng (**api_views.py**) chỉ là để các mã nguồn được tường minh, dễ quản lý hơn.

2. Thêm đoạn mã sau vào tập tin **api_views.py**

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Book

@api_view()
def first_api_view(request):
    num_books = Book.objects.count()
    return Response({"num_books": num_books})
```

   Dòng đầu tiên import decorator **api_view** để chuyển view thành hàm API theo DRF và dòng thứ hai import lớp **Response** dùng để trả về kết quả cho API.

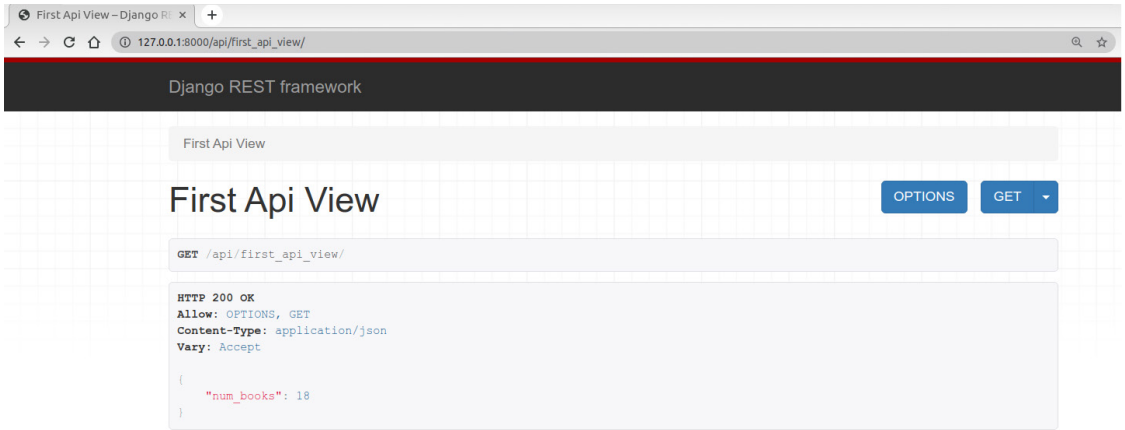
   Hàm view trả về một đối tượng **Response** chứa một *dictionary* có số lượng cuốn sách trong CSDL (**num_books**).

3. Mở tập tin **books/reviews/urls.py** và import mô đun **api_views**. Sau đó, thêm một đường dẫn URL mới ánh xạ đến **api_views** đã định nghĩa ở **Bước 1**.

```
from . import views, api_views

urlpatterns = [
    path('api/first_api_view/',  api_views.first_api_view),
    …
]
```

4. Khởi chạy trình chủ Django và mở trang http://127.0.0.1:8000/api/first_api_view/ để gửi yêu cầu API. Kết quả sẽ hiển thị như trong **Hình I-1**.
   Khi truy xuất URL ở trên, một thông điệp **HTTPRequest** với phương thức là **GET** được gửi tới điểm cuối API (API endpoint) và giá trị trả về là dictionary theo định dạng JSON chứa cặp key-value **{"num_books": 18}.** DRF cung cấp một giao diện tương đối đẹp mắt để hiển thị kết quả trả về khi gửi yêu cầu đến điểm cuối API bằng trình duyệt.



Hình I-1. View API với kết quả trả về là số cuốn sách trong CSDL

5. Khi ta gửi yêu cầu đến điểm cuối API bằng lệnh curl thì kết quả trả về chỉ gồm từ điển chứa dữ liệu:

```
curl.exe http://127.0.0.1:8000/api/first_api_view/
{"num_books":18}
```

II. Tạo một view API để hiển thị danh sách các cuốn sách

Trong bài thực hành này, ta sẽ sử dụng bộ tuần tự hóa dữ liệu để tạo một API trả về danh sách tất cả các cuốn sách.

1. Tạo một tập tin tên là **serializers.py** trong thư mục **bookr/reviews**. Ta sẽ thêm vào tập tin này đoạn mã để tuần tự hóa dữ liệu phục vụ cho API.
2. Thêm đoạn mã sau vào tập tin **serializers.py**:

```
from rest_framework import serializers

class PublisherSerializer(serializers.Serializer):
    name = serializers.CharField()
    website = serializers.URLField()
    email = serializers.EmailField()

class BookSerializer(serializers.Serializer):
    title = serializers.CharField()
    publication_date = serializers.DateField()
    isbn = serializers.CharField()
    publisher = PublisherSerializer()
```

Trong này, dòng đầu tiên **import** gói **serializers** phục vụ cho tuần tự hóa dữ liệu.

Sau đó ta định nghĩa 2 lớp là **PublisherSerializer** và **BookSerializer**. Hai lớp này là các bộ tuần tự hóa cho 2 model **Publisher** và **Book**. Cả 2 bộ tuần tự hóa đều là lớp con của lớp **serializers.Serializer**. Đồng thời ta cũng định nghĩa kiểu dữ liệu cho các thuộc tính của lớp sử dụng các lớp tương ứng của gói **serializers** như *CharField*, *EmailField*,…

Các thuộc tính trong các bộ tuần tự này cũng tương ứng với các thuộc tính mà ta đã định nghĩa trong các lớp model **Publisher** và **Book**, như *name, website, email* trong lớp **Publisher** và *title, publication_date, isbn* trong lớp **Book**. Trong lớp **Book** còn có thuộc tính **publisher** là khóa ngoại liên kết với model **Publisher** nên ta sử dụng kiểu dữ liệu **PublisherSerializer** cho thuộc tính này.

3. Mở tập tin **bookr/reviews/api_views/py** và xóa tất cả các dòng mã hiện có thay thế bằng đoạn mã sau:

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Book
from .serializers import BookSerializer

@api_view()
def all_books(request):
    books = Book.objects.all()
    book_serializer = BookSerializer(books, many=True)
    return Response(book_serializer.data)
```

Ở dòng mã thứ tư , ta **import** lớp tuần tự hóa **BookSerializer** đã định nghĩa ở Bước trước.

Sau đó, ta thêm một view **all_books** (dạng hàm). View này truy xuất danh sách tất cả các cuốn sách từ CSDL và tuần tự hóa dữ liệu này sử dụng **BookSerializer**. Tham số **many=True** để chỉ ra tằng đối tượng **books** ở đây là một **queryset** hay là một danh sách

nhiều đối tượng. Lưu ý là việc tuần tự hóa dữ liệu sẽ nhận vào một đối tượng Python và trả về đoạn dữ liệu đã tuần tự hóa theo định dạng JSON. Kết quả trả về có dạng như sau:

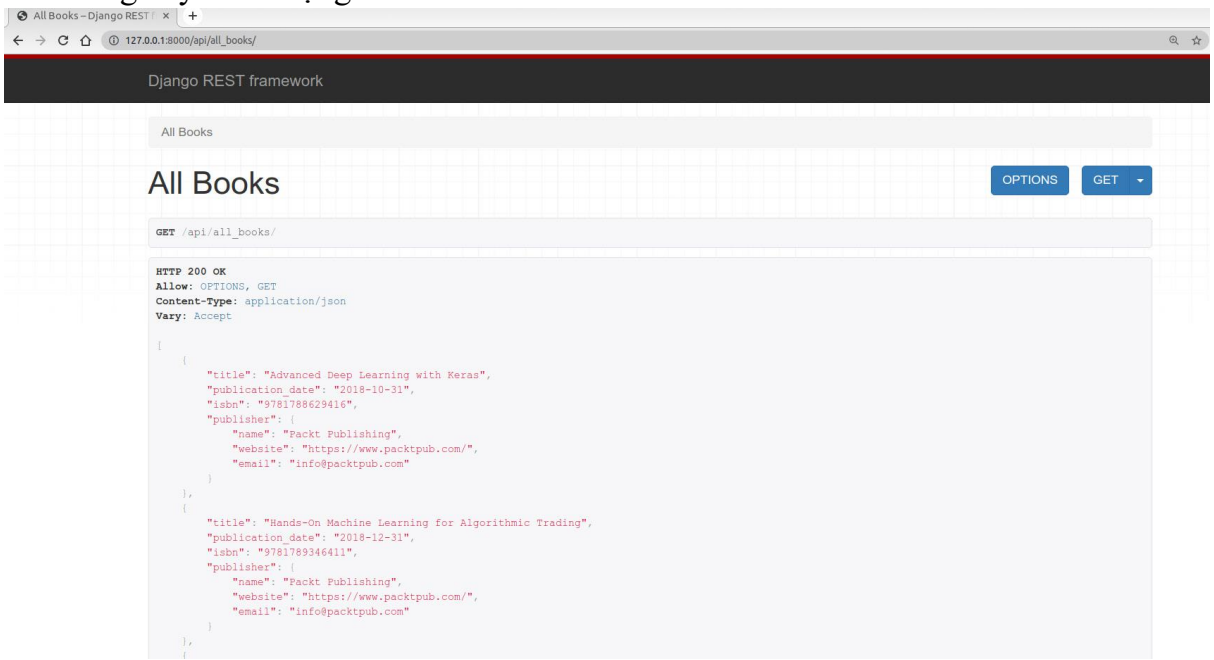
```
[OrderedDict([('title', 'Advanced Deep Learning with Keras'),
('publication_date', '2018-10-31'), ('isbn', '9781788629416'),
('publisher', OrderedDict([('name', 'Packt Publishing'),
('website', 'https://www.packtpub.com/'), ('email', 'info@
packtpub.com')]))]), OrderedDict([('title', 'Hands-On Machine
Learning for Algorithmic Trading'), ('publication_date',
'2018-12-31'), ('isbn', '9781789346411'), ('publisher',
OrderedDict([('name', 'Packt Publishing'), ('website', 'https://
www.packtpub.com/'), ('email', 'info@packtpub.com')]))]) …
```

4. Mở tập tin **bookr/reviews/urls.py**, xóa bỏ đoạn khai báo đường dẫn URL đến **first_api_view** (đã định nghĩa trong bài thực hành I) và thêm vào một đường dẫn URL đến API **all_books**:

```
from django.urls import path
from . import views, api_views

urlpatterns = [
    path(
        'api/all_books/',
        api_views.all_books,
        name='all_books'
    ),
    …
]
```

5. Chạy trình chủ Django và truy cập đến trang http://127.0.0.1:8000/api/all_books/, giao diện của trang này sẽ có dạng như **Hình II-1**.



Hình II-1. Danh sách các cuốn sách trả về khi gọi API all_books

III. Tạo view API dựa trên lớp và sử dụng ModelSerializer

Trong bài này, ta sẽ tạo một view API dựa trên lớp để liệt kê tất cả các cuốn sách trong CSDL sử dụng **ModelSerializer**.

1. Mở tập tin **bookr/reviews/serializers.py**, xóa đoạn mã hiện có trong tập tin và thay bằng đoạn mã sau:

```
from rest_framework import serializers
from .models import Book, Publisher

class PublisherSerializer(serializers.ModelSerializer):

    class Meta:
        model = Publisher
        fields = ['name', 'website', 'email']

class BookSerializer(serializers.ModelSerializer):
    publisher = PublisherSerializer()

    class Meta:
        model = Book
        fields = ['title','publication_date','isbn','publisher']
```

   Ở đây, ta khai báo 2 lớp tuần tự hóa dữ liệu **PublisherSerializer** và **BookSerializer** kế thừa từ lớp cha **serializers.ModelSerializer**. Ta không cần thiết phải đặc tả từng trường cần tuần tự hóa dữ liệu nữa mà chỉ cần khai báo danh sách các trường này qua thuộc tính **fields**, kiểu dữ liệu của các trường sẽ dựa trên định nghĩa của các lớp model tương ứng.

   Mặc dù khai báo các trường trong thuộc tính **fields** là đủ nhưng trong các trường hợp đặc biệt, như trường hợp trường **publisher** là khóa ngoại liên kết với lớp **Publisher**, thì ta phải khai báo lớp **PublisherSerializer** để tuần tự hóa dữ liệu cho trường **publisher**.

2. Tiếp theo, mở tập tin **bookr/reviews/api_views.py**, xóa bỏ toàn bộ đoạn mã hiện có trong tập tin và thêm vào đoạn mã sau:

```
from rest_framework import generics
from .models import Book
from .serializers import BookSerializer

class AllBooks(generics.ListAPIView):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

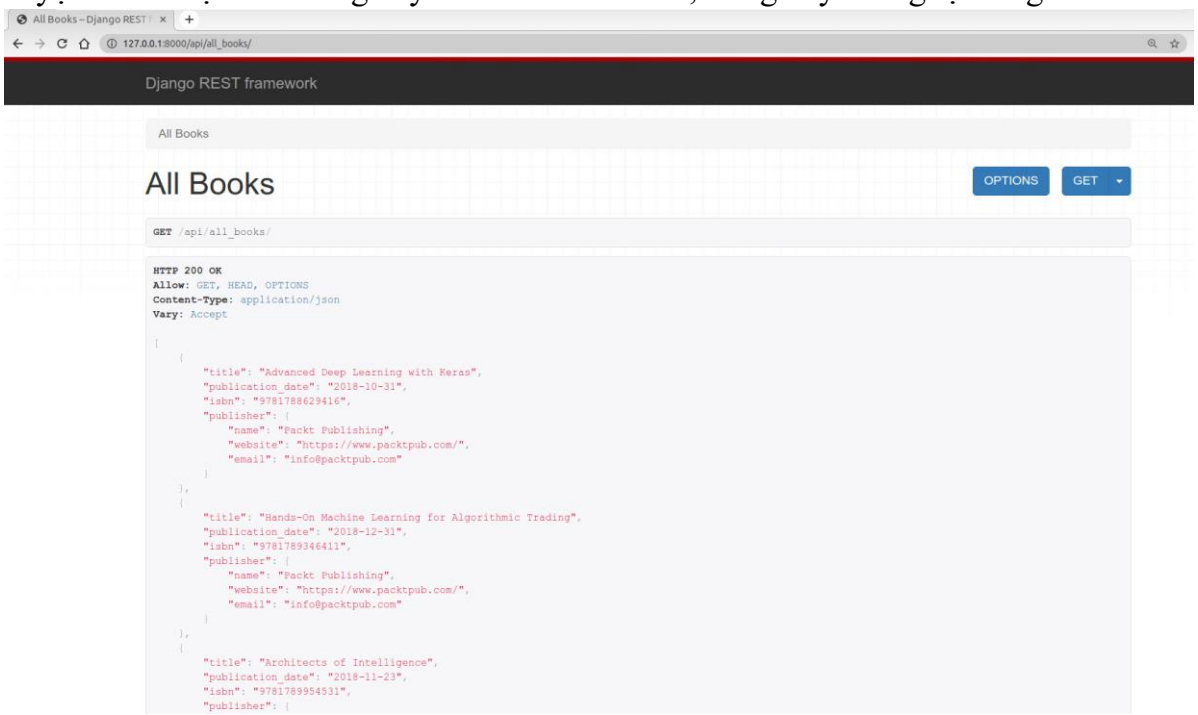
   Ở đây, ta định nghĩa view dựa trên lớp kế thừa từ lớp **ListAPIView**. Điều này có nghĩa là danh sách các cuốn sách được định nghĩa như là một thuộc tính của lớp (queryset) và ta không phải viết hàm để kiểm soát trực tiếp **request** cũng như gọi bộ tuần tự hóa dữ liệu. Lớp **BookSerializer** khai báo ở **Bước 1** cũng được **import** và gán thành một thuộc tính của lớp này (**serializer_class**).

   Mở tập tin **bookr/reviews/urls.py** và điều chỉnh khai báo đường dẫn URL của API **/api/all_books** để ánh xạ đến view **AllBooks** vừa tạo:

```
urlpatterns = [
    path(
        'api/all_books/',
        api_views.AllBooks.as_view(),
        name='all_books'
    )
    …
]
```

Lưu ý là khi sử dụng view dựa trên lớp thì ta phải thêm phương thức **as_view()** vào sau tên lớp (**AllBooks**).

3. Chạy trình chủ Django và mở trang API http://127.0.0.1:8000/api/all_books/ trên trình duyệt. Giao diện của trang này sẽ như **Hình III-1**, trang này tương tự trang ở **Hình II-1**.



Hình III-1. Trang API danh sách các cuốn sách tại điểm cuối all_books

IV. Sử dụng Viewset và router

Trong bài thực hành này, ta sẽ kết hợp các view hiện có để tạo thành một viewset và tạo định tuyến phù hợp đến viewset này.

1. Mở tập tin **bookr/reviews/serializers.py**, xóa đoạn mã đang có trong tập tin và thêm vào đoạn mã sau:

```
from django.contrib.auth.models import User
from django.utils import timezone
from rest_framework import serializers
from rest_framework.exceptions import NotAuthenticated,
PermissionDenied
from .models import Book, Publisher, Review
from .utils import average_rating

class PublisherSerializer(serializers.ModelSerializer):

    class Meta:
        model = Publisher
        fields = ['name', 'website', 'email']

class UserSerializer(serializers.ModelSerializer):

    class Meta:
        model = User
        fields = ['username', 'email']

class ReviewSerializer(serializers.ModelSerializer):
    creator = UserSerializer(read_only=True)
    book = serializers.StringRelatedField(read_only=True)

    class Meta:
        model = Review
        fields = ['pk', 'content', 'date_created',
'date_edited', 'rating', 'creator', 'book', 'book_id']

    def create(self, validated_data):
        request = self.context["request"]
        creator = request.user
        if not creator.is_authenticated:
            raise NotAuthenticated('Authentication required.')
        book = Book.objects.get(pk=request.data['book_id'])
        return
Review.objects.create(content=validated_data['content'],
book=book, creator=creator, rating=validated_data['rating'])

    def update(self, instance, validated_data):
        request = self.context['request']
        creator = request.user
        if not creator.is_authenticated or instance.creator_id
!= creator.pk:
```

```
            raise PermissionDenied('Permission denied, you are
not the creator of this review')
        instance.content = validated_data['content']
        instance.rating = validated_data['rating']
        instance.date_edited = timezone.now()
        instance.save()
        return instance

class BookSerializer(serializers.ModelSerializer):
    publisher = PublisherSerializer()
    rating = serializers.SerializerMethodField('book_rating')
    reviews = serializers.SerializerMethodField('book_reviews')

    def book_rating(self, book):
        reviews = book.review_set.all()
        if reviews:
            return average_rating([review.rating for review in
reviews])
        else:
            None

    def book_reviews(self, book):
        reviews = book.review_set.all()
        if reviews:
            return ReviewSerializer(reviews, many=True).data
        else:
            None

    class Meta:
        model = Book
        fields = ['title', 'publication_date', 'isbn',
'publisher', 'rating', 'reviews']
```

Trong đoạn mã này, ta thêm 2 trường vào lớp **BookSerializer** là **reviews** và **rating**. Logic xử lý 2 trường này được định nghĩa bằng 2 phương thức tương ứng ở trong cùng lớp là **book_rating()** và **book_reviews().**

2. Mở tập tin **bookr/reviews/api_views.py**, xóa đoạn mã hiện có và thay bằng đoạn mã sau:

```
from rest_framework import viewsets
from rest_framework.pagination import (LimitOffsetPagination)
from .models import Book, Review
from .serializers import (BookSerializer,ReviewSerializer)

class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer

class ReviewViewSet(viewsets.ModelViewSet):
    queryset = Review.objects.order_by('-date_created')
    serializer_class = ReviewSerializer
```

```
    pagination_class = LimitOffsetPagination
    authentication_classes = []
```

Ở đây, ta đã xóa 2 view **AllBook** và **BookDetail**, và thay bằng **BookViewSet** và **ReviewVietSet**. Ở dòng đầu tiên, ta **import** gói **ViewSets** từ DRF. Lớp **BookViewSet** là lớp con của lớp **ReadOnlyModelViewSet** để đảm bảo rằng view này chỉ được dùng cho thao tác đọc dữ liệu ra (GET).

3. Tiếp theo, mở tập tin **bookr/reviews/urls.py**, xóa bỏ 2 dòng khai báo ánh xạ URL đầu tiên có đường dẫn bắt đầu bằng **api/** và thêm vào các dòng mã (tô đậm) sau:

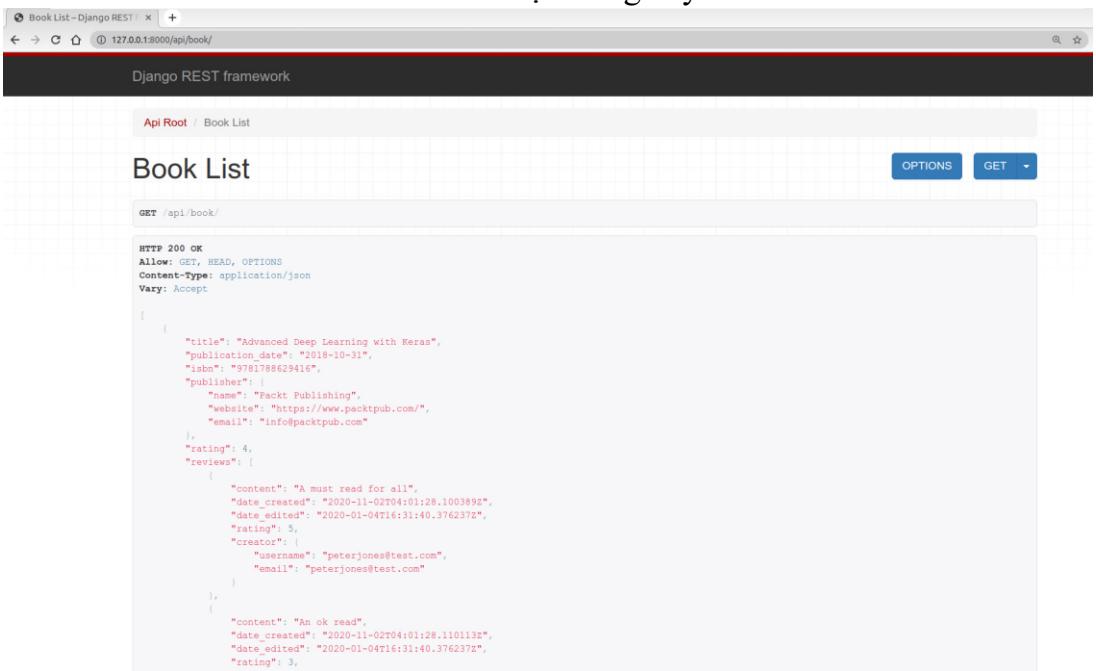
```
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from . import views, api_views
router = DefaultRouter()
router.register(r'books', api_views.BookViewSet)
router.register(r'reviews', api_views.ReviewViewSet)

urlpatterns = [
    path('api/', include((router.urls, 'api'))),
    …
]
```

Ở đây ta kết hợp 2 đường dẫn URL **all_books** và **book_detail** thành một đường dẫn duy nhất là **books**. Ta cũng thêm một điểm cuối API mới nằm trong đường dẫn **reviews**.

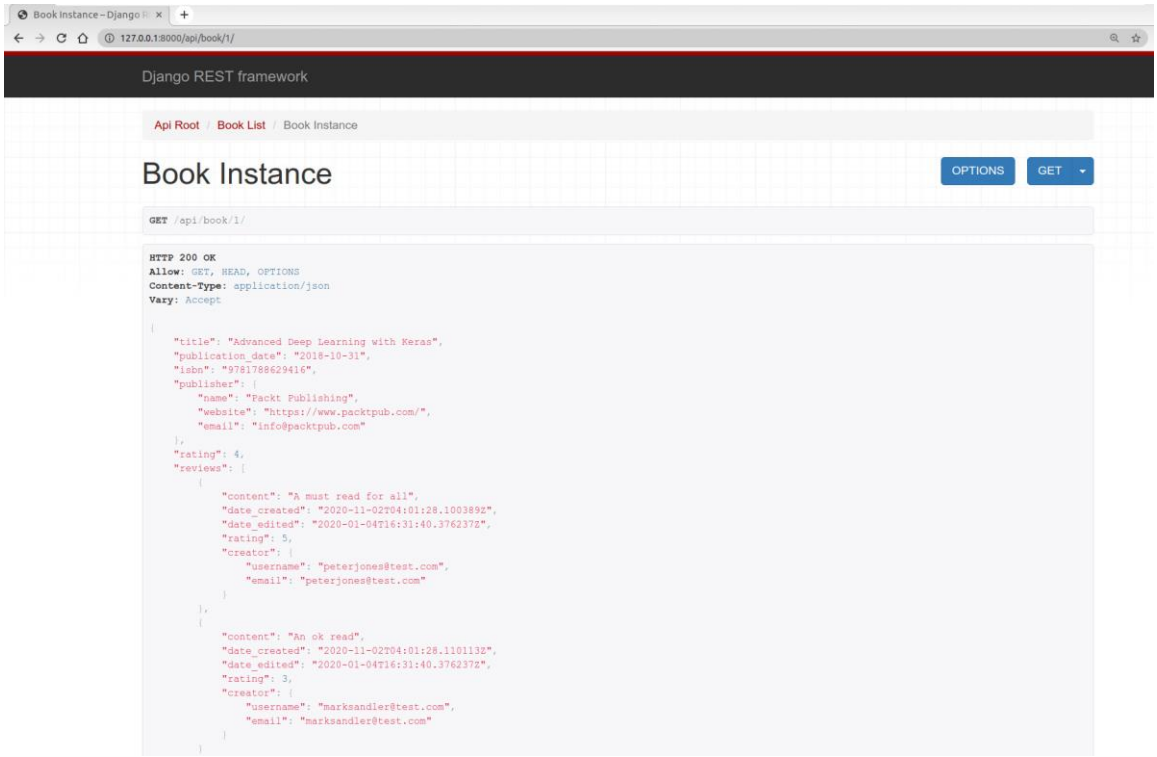
Ta bắt đầu bằng cách **import** lớp **DefaultRouter** từ gói **rest_framework.routers**. Sau đó, ta tạo một đối tượng **router** sử dụng lớp **DefaultRouter** và đăng ký 2 view (dạng lớp) **BookViewSet** và **ReviewViewSet**. Điều này đảm bảo rằng **BookViewSet** sẽ được gọi khi có yêu cầu đến API có đường dẫn **/api/books.**

4. Khởi chạy trình chủ Django và truy cập đến trang http://127.0.0.1:8000/api/books/ để lấy ra danh sách tất cả các cuốn sách. Giao diện trang này sẽ như **Hình IV-1**.



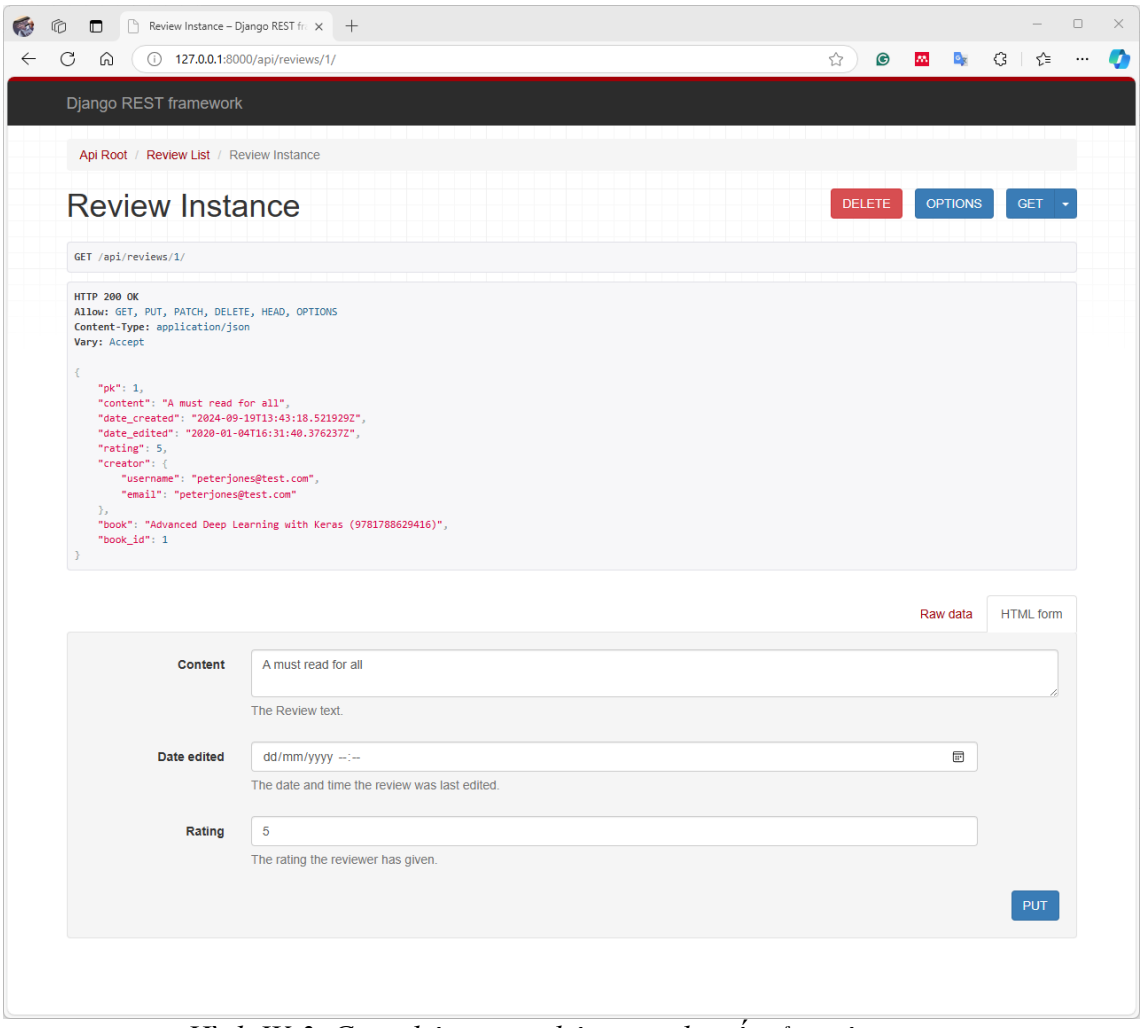
Hình IV-1. Danh sách các cuốn sách khi gọi API theo đường dẫn /api/books

5. Ta cũng có thể truy cập để xem chi tiết một cuốn sách cụ thể bằng cách thêm ID của cuốn sách vào sau. Ví dụ: http://127.0.0.1:8000/api/books/1/ (ID=1). Kết quả trả về sẽ như **Hình IV-2**.



Hình IV-2. Giao diện trang thông tin chi tiết của một cuốn sách

6. Khi ta truy cập thông tin một review, ví dụ: http://127.0.0.1:8000/api/reviews/1/, kết quả trả về sẽ như **Hình IV-3**. Trong hình này, ngoài phần thông tin của review còn có nút Delete để xóa review và form để cập nhật, chỉnh sửa review.



Hình IV-3. Giao diện trang thông tin chi tiết của một review

V. Triển khai xác thực sử dụng token

Trong bài thực hành này, ta sẽ triển khai xác thực dựa trên token cho các API của ứng dụng **Bookr**.

1. Mở tập tin **bookr/settings.py** và thêm app **rest_framework.authtoken** vào biến **INSTALLED_APPS**:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'rest_framework.authtoken',
    'reviews'
]
```

2. Do app **authtoken** liên kết với các thay đổi trong CSDL nên ta cần chạy lại lệnh **migrate**:

```
python manage.py migrate
```

3. Mở tập tin **bookr/reviews/api_views.py**, xóa bỏ đoạn mã hiện có và thay bằng đoạn mã sau:

```
from django.contrib.auth import authenticate
from rest_framework import viewsets
from rest_framework.authentication import TokenAuthentication
from rest_framework.authtoken.models import Token
from rest_framework.pagination import LimitOffsetPagination
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework.status import HTTP_404_NOT_FOUND,
HTTP_200_OK
from rest_framework.views import APIView

from .models import Book, Review
from .serializers import BookSerializer, ReviewSerializer

class Login(APIView):

    def post(self, request):
        user = authenticate(username=request.data.get(
"username"), password=request.data.get("password"))
        if not user:
            return Response({'error': 'Credentials are incorrect
or user does not exist'}, status=HTTP_404_NOT_FOUND)
        token, _ = Token.objects.get_or_create(user=user)
        return Response({'token': token.key},status=HTTP_200_OK)

class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

```
    authentication_classes = []
    permission_classes = []

class ReviewViewSet(viewsets.ModelViewSet):
    queryset = Review.objects.order_by('-date_created')
    serializer_class = ReviewSerializer
    pagination_class = LimitOffsetPagination
    authentication_classes = []
```

Trong đoạn mã này, ta định nghĩa một view tên là **Login**. Mục đích của view này là cho phép người dùng có thể nhận một token để họ có thể sử dụng cho xác thực các API.

Ta ghi đè phương thức **post()** của view này để thay đổi cách thức xử lý khi người dùng gửi dữ liệu đến ứng dụng. Trước tiên, ta sử dụng phương thức **authenticate**() từ thư viện **auth** của Django để kiểm tra **username** và mật khẩu có đúng không. Nếu là đúng thì ta sẽ có đối tượng **user**, nếu không thì một thông báo lỗi **HTTP 404**. Nếu ta có đối tượng **user** hợp lệ thì ta sẽ tạo hoặc lấy một **token** bằng phương thức **Token.objects.get_or_create()** và trả về cho người dùng.

4. Tiếp theo, ta thêm lớp xác thực vào **BookViewSet**. Điều này có nghĩa là khi một người dùng truy cập vào viewset này, ứng dụng sẽ yêu cầu họ phải xác thực sử dụng phương thức xác thực dựa trên token. Lưu ý là ta có thể thêm một danh sách các phương thức xác thực có thể được chấp nhận (ngoài phương thức dựa trên token). Ta cũng thêm thuộc tính **permissions_classes** (với giá trị là lớp **IsAuthenticated** có sẵn của DRF) chỉ ra lớp sẽ thực hiện kiểm tra người dùng đang đăng nhập có đủ quyền để truy cập view không.

```
class BookViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    authentication_classes = [TokenAuthentication]
    permission_classes = [IsAuthenticated]
```

5. Mở tập tin **bookr/reviews/urls.py** và thêm vào đường dẫn URL sau:

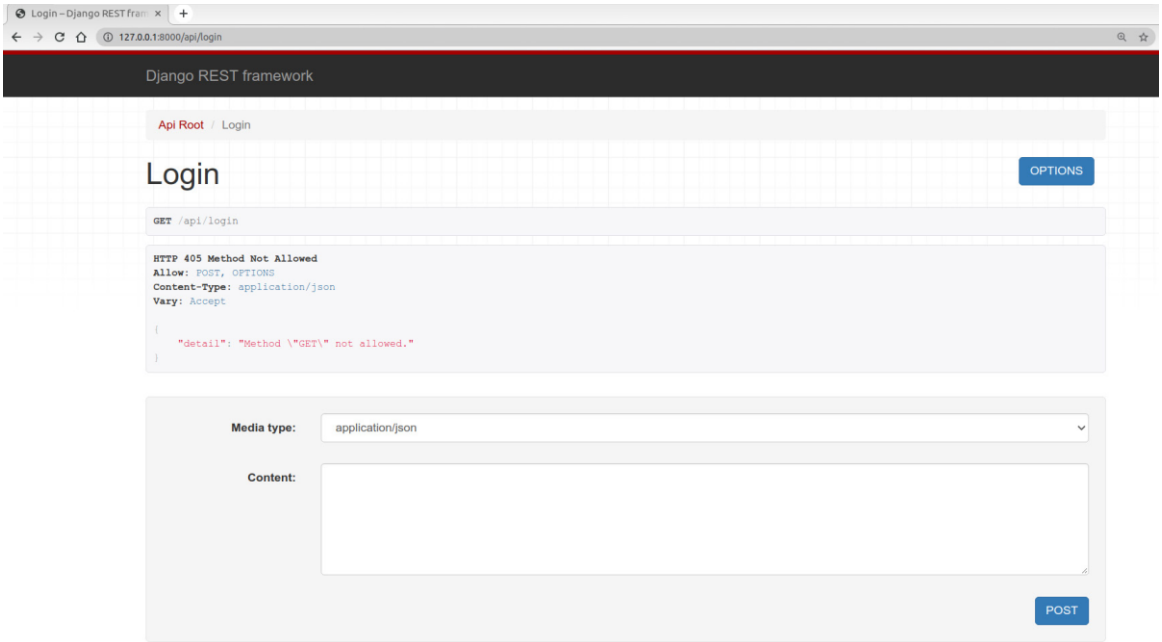
```
urlpatterns = [
    …
    path('api/login', api_views.Login.as_view(), name='login')
    …
]
```

6. Khởi chạy trình chủ Django và truy cập trang http://127.0.0.1:8000/api/login. Giao diện tra này sẽ như **Hình V-1**.
   Trang /**api/login** chỉ chấp nhận yêu cầu từ gói tin POST nên xuất hiện thông báo lỗi "**Method GET not allowed**".

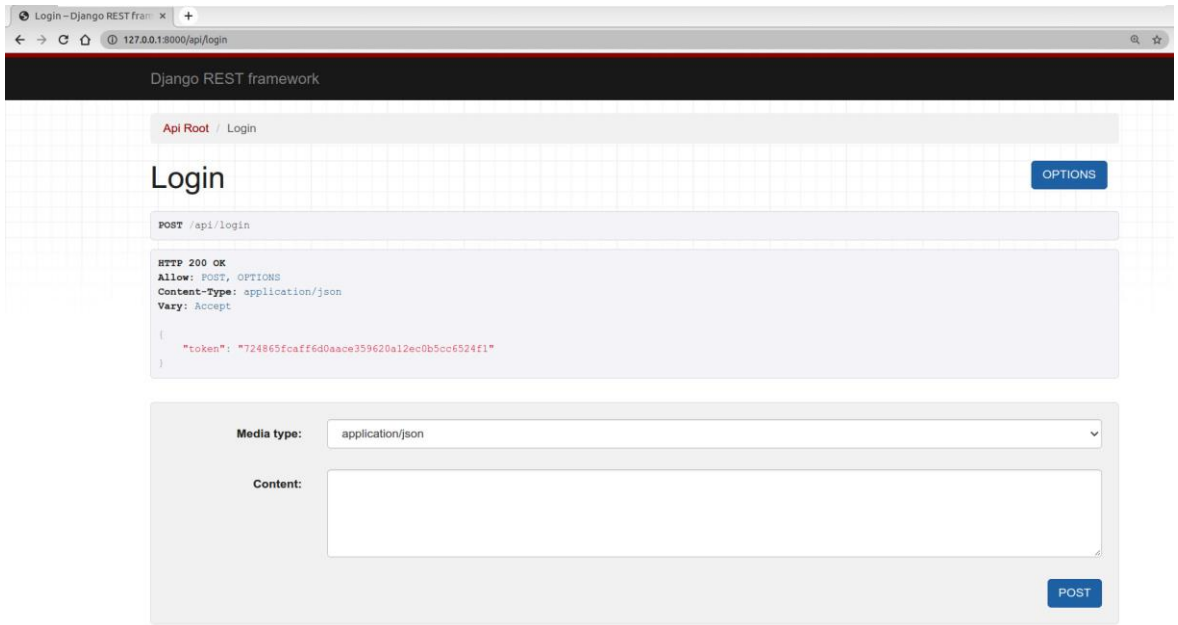
7. Tiếp theo, ta thêm đoạn mã sau vào hộp nhập **Content** của trang **Login** và nhấn **POST**:

```
{
"username": "Peter",
"password": "testuserpassword"
}
```

Ta sẽ cần thay thế giá trị **username** và **password** bằng username và mật khẩu thực tế của tài khoản ta có trong CSDL. Trang **Login** sau khi nhấn nút **POST** (và cung cấp đúng username/ password) sẽ như **Hình V-2**. Sao chép lại giá trị token này để dùng về sau.



Hình V-1. Trang Login



Hình V-2. Token trả về sau khi đăng nhập thành công

8. Thử truy cập đến API liệt kê danh sách sách qua URL http://127.0.0.1:8000/api/books/. Do ta chưa đăng nhập nên sẽ hiện thông báo như **Hình V-3.**
   Ta cũng nhận được lỗi tương tự khi dùng lệnh curl:

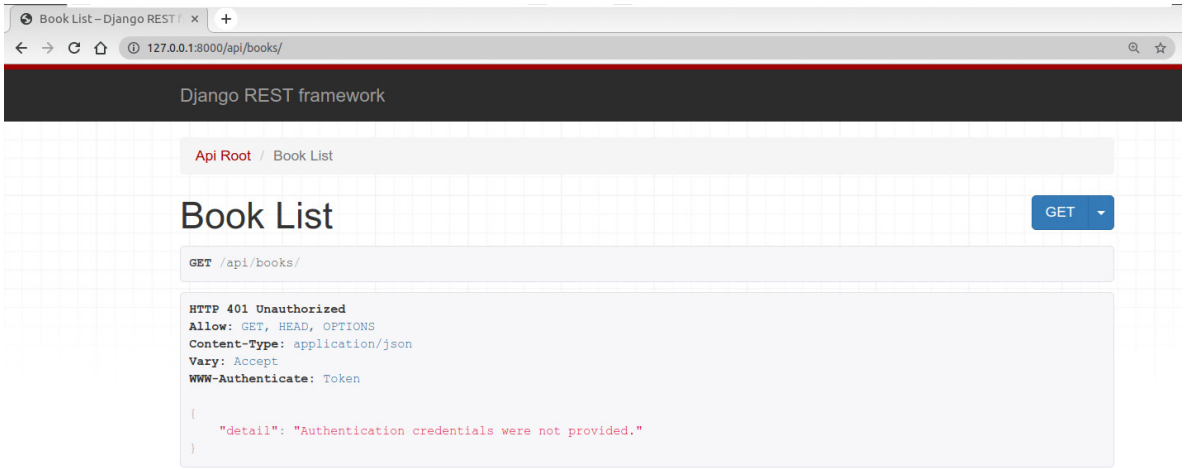
```
curl -X GET http://127.0.0.1:8000/api/books/
{"detail":"Authentication credentials were not provided."}
```

   Nếu ta truyền vào token (đã nhận được ở bước 7) thì có thể nhận được kết quả:

```
curl -X GET http://127.0.0.1:8000/api/books/ -H "Authorization:
Token cd5dafa1d4361fd1502652d266eed3dcdb55c64f1"
[{"title":"Advanced Deep Learning with Keras","publication_
date":"2018-10-31","isbn":"9781788629416","publish
er":{"name":"Packt Publishing","website":"https://
www.packtpub.com/","email":"info@packtpub.
com"},"rating":4,"reviews":[{"content":"A must read for
all","date_created":…
```

Điều này đảm bảo rằng chỉ những người đã đăng nhập mới nhận được kết quả từ lời gọi API.

Lưu ý: *Ngoài sử dụng lệnh curl, ta cũng có thể sử dụng các công cụ kiểm tra API như Postman để kiểm thử các API có yêu cầu xác thực dùng Token.*



Hình V-3. Thông điệp yêu cầu phải cung cấp thông tin xác thực